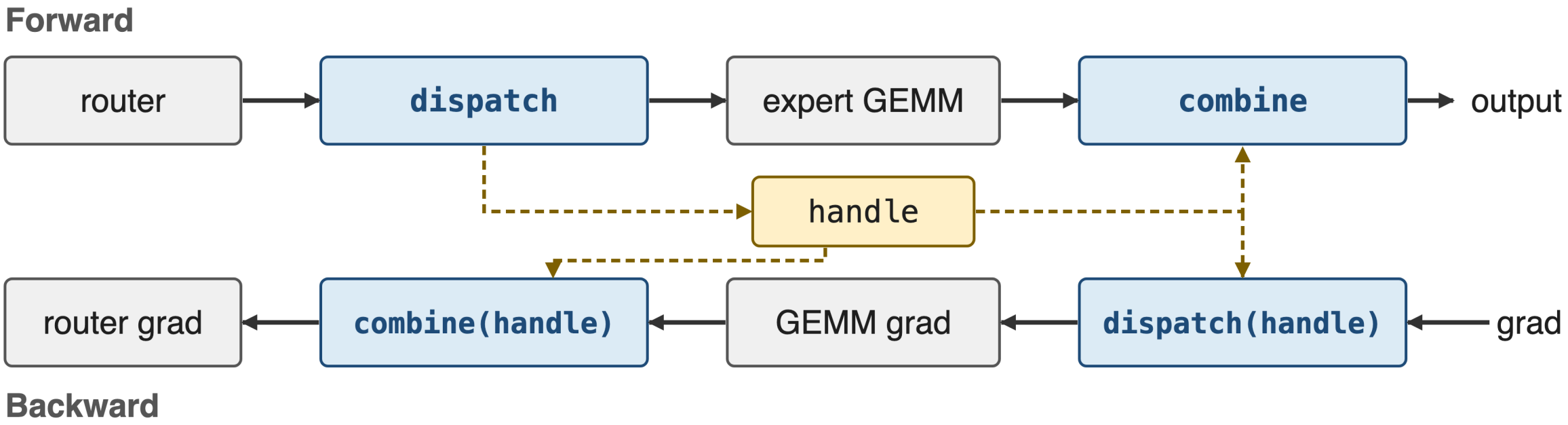
EPBuffer + analytic SM/QP budget

One EP API for train, prefill and decode; SMs computed, not tuned

DeepSeek-AI · DeepEP V2 (ElasticBuffer) and V2.5 (EPBuffer)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Every MoE layer calls dispatch and combine twice



Same calls for training, prefill and decode; backward reuses the forward routing handle

V1 spent 24 SMs per GPU on communication

24 SMs

per GPU for V3-like training, set by hand, plus a chunk-size table that covers only 10 EP sizes

DeepEP V2 release notes (README at b306af0, PR #605); V1 README example and deep_ep/buffer.py at v1.2.1

V1 looked its chunk sizes up in a table

- One global SM count: 20 by default
- 5 knobs per EP size, "TODO: automatically tune"
- Decode API apart: QPs = local experts

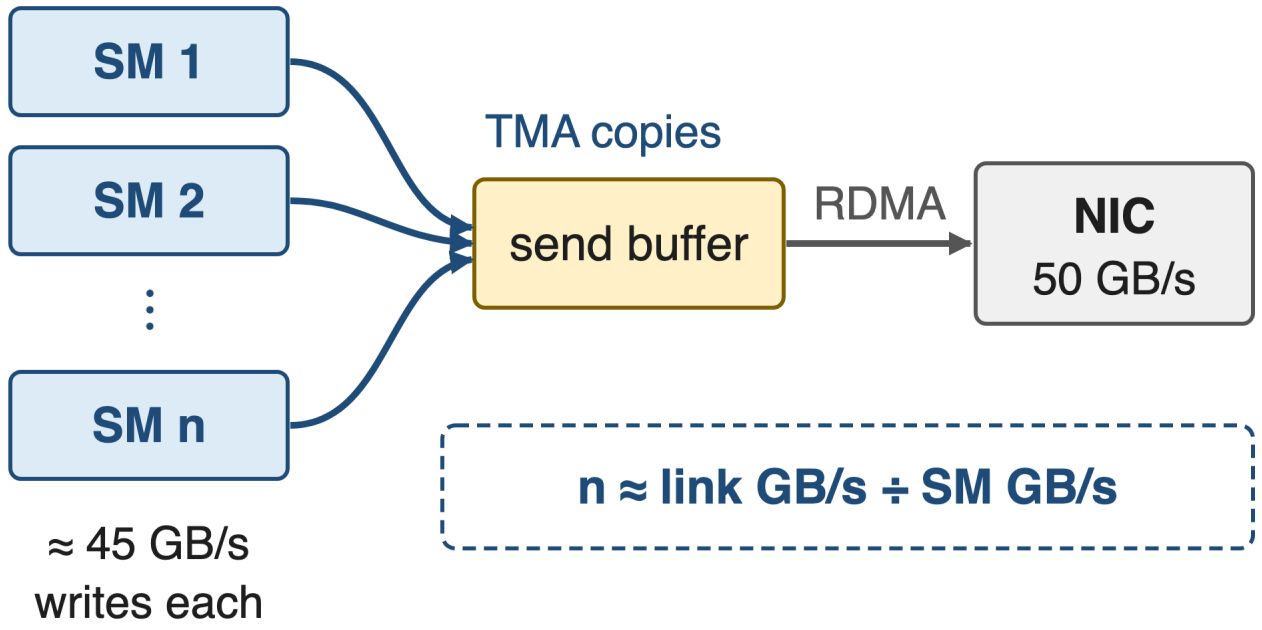
EP size	NVLink chunk send / recv	RDMA chunk send / recv
8	6 / 256	6 / 128
16	36 / 288	20 / 128
32	32 / 288	32 / 128
64	20 / 288	28 / 128
128	20 / 560	32 / 128

⋯ 10 hand-picked sizes in total ⋯

48	**assert: unsupported EP size**

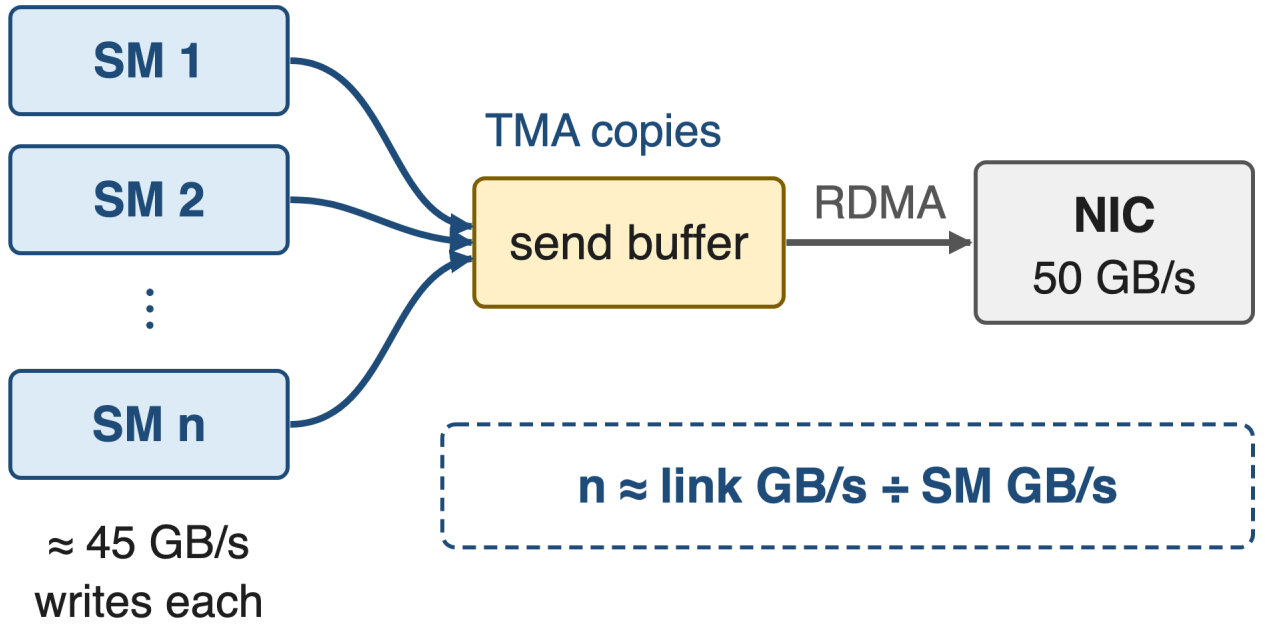
Insight: SMs only need to feed the slowest link

- One SM writes ≈ 45–50 GB/s to HBM
- A 400 Gb/s NIC drains 50 GB/s
- Routing fixes the bytes per token



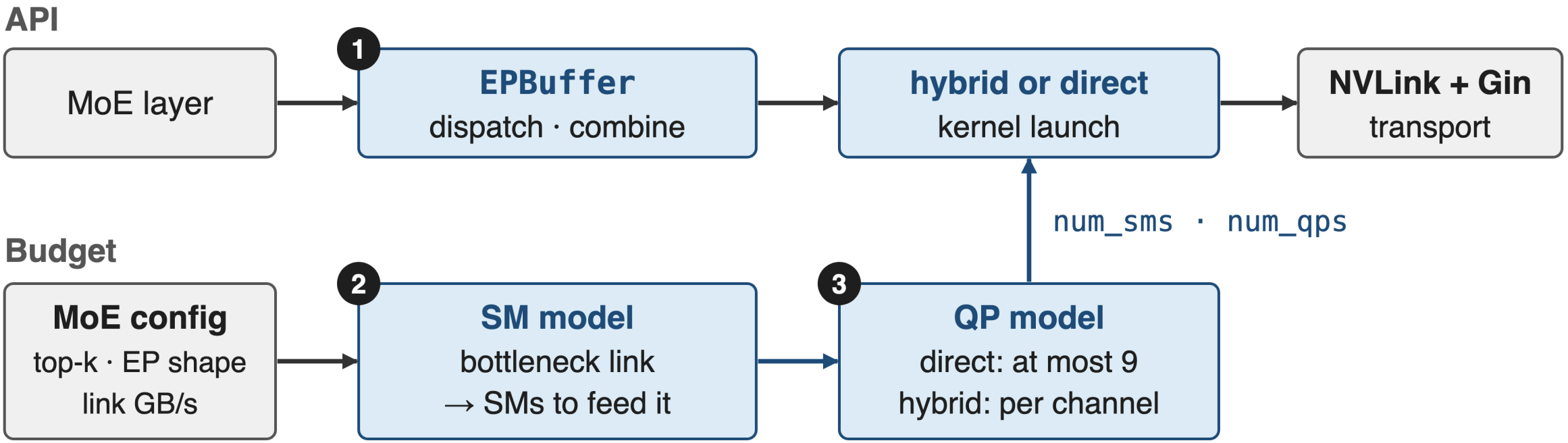
Insight: SMs only need to feed the slowest link

- One SM writes ≈ 45–50 GB/s to HBM
- A 400 Gb/s NIC drains 50 GB/s
- Routing fixes the bytes per token



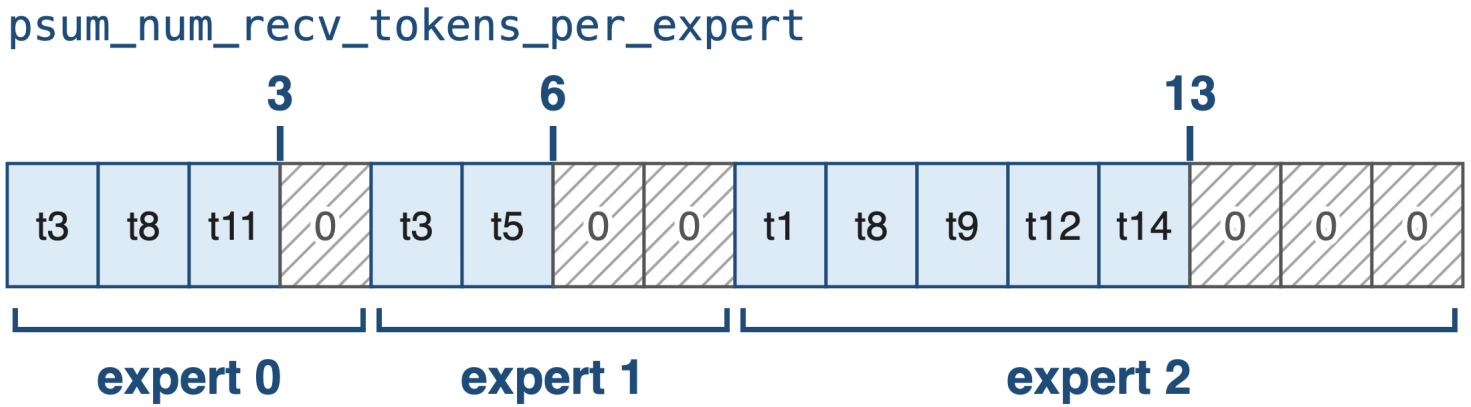
So the SM budget is small, and it can be computed

One buffer, one computed budget



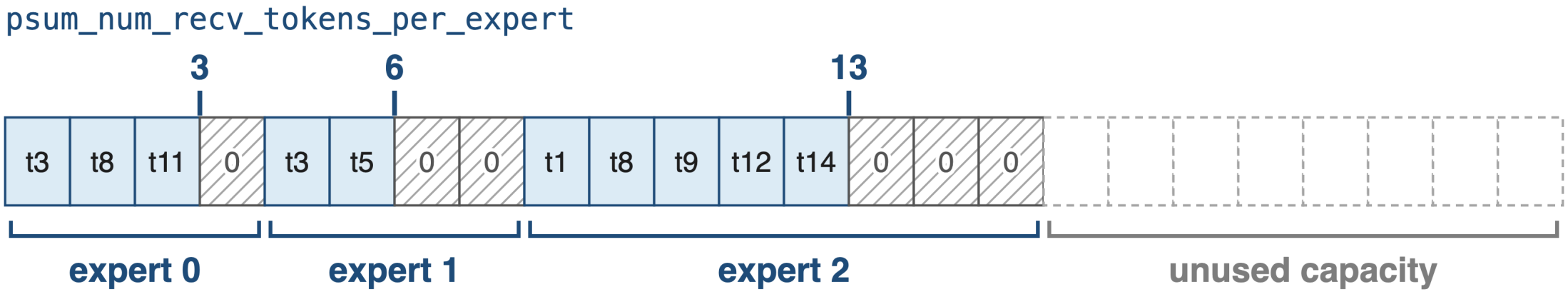
❶ unified API · ❷ SM model · ❸ QP model; any call can still override num_sms / num_qps

❶ One dispatch layout serves training and decode



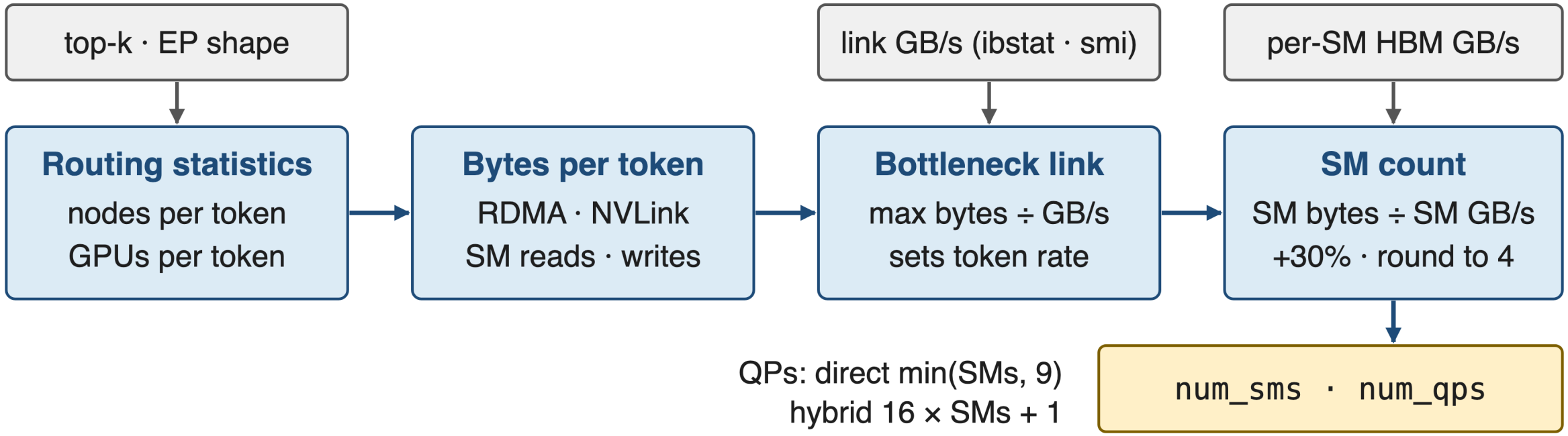
Train / prefill (CPU sync): one row per token–expert pair, padded to the GEMM alignment, exact row count

❶ One dispatch layout serves training and decode



Decode (no CPU sync): rows sized to capacity; the grouped GEMM reads valid ranges from the GPU-side psum

❷ The SM count follows the bottleneck link



Balanced-gate model in Python; computed once per buffer and cached

❷ Worked example: EP 8×4 needs 6 SMs

❶ Routing statistics

top-8 of 256 experts

7.27 GPUs

on 3.61 nodes
per token

❷ Worked example: EP 8×4 needs 6 SMs

❶ Routing statistics

top-8 of 256 experts

7.27 GPUs

on 3.61 nodes
per token

❷ Bottleneck link

RDMA 0.37 B ÷ 50 GB/s
NVLink 0.875 B ÷ 160 GB/s

134 GB/s

delivered · RDMA-bound

❷ Worked example: EP 8×4 needs 6 SMs

❶ Routing statistics

top-8 of 256 experts

7.27 GPUs

on 3.61 nodes
per token

❷ Bottleneck link

RDMA 0.37 B ÷ 50 GB/s
NVLink 0.875 B ÷ 160 GB/s

134 GB/s

delivered · RDMA-bound

❸ SM count

1.26 B SM writes per byte
169 GB/s ÷ 50 GB/s = 3.4

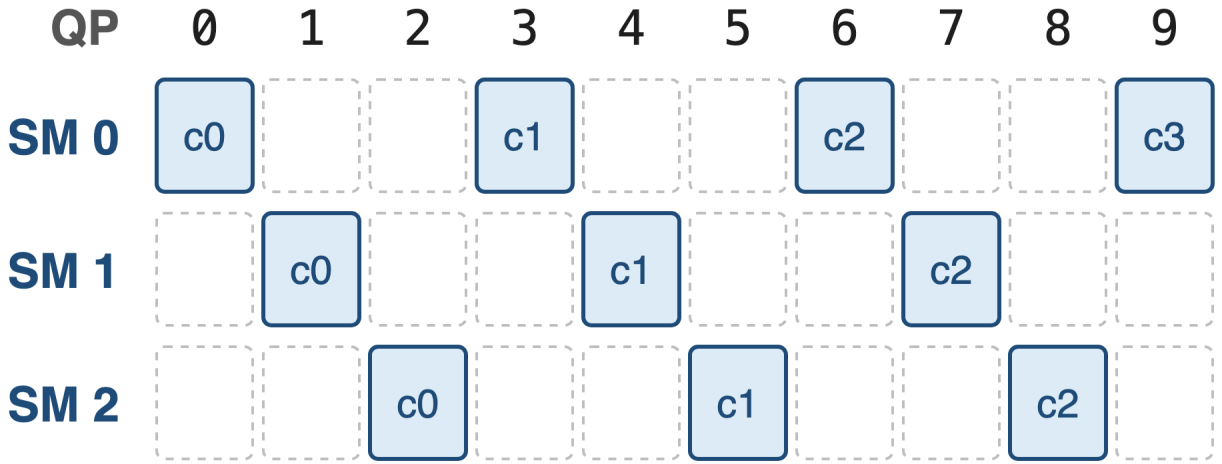
6 SMs

after +25% and rounding

Matches the 6 SMs of the release table's EP 8×4 row (derived, V2.0 constants)

❸ QPs follow SMs and channels

- Direct: min(SMs, 9) QPs, fewer doorbells
- Hybrid: 16 × SMs + 1, one per channel
- Allocated once: 129, or 65 on MT4131 NICs



V3-style training drops from 24 to 4–6 SMs

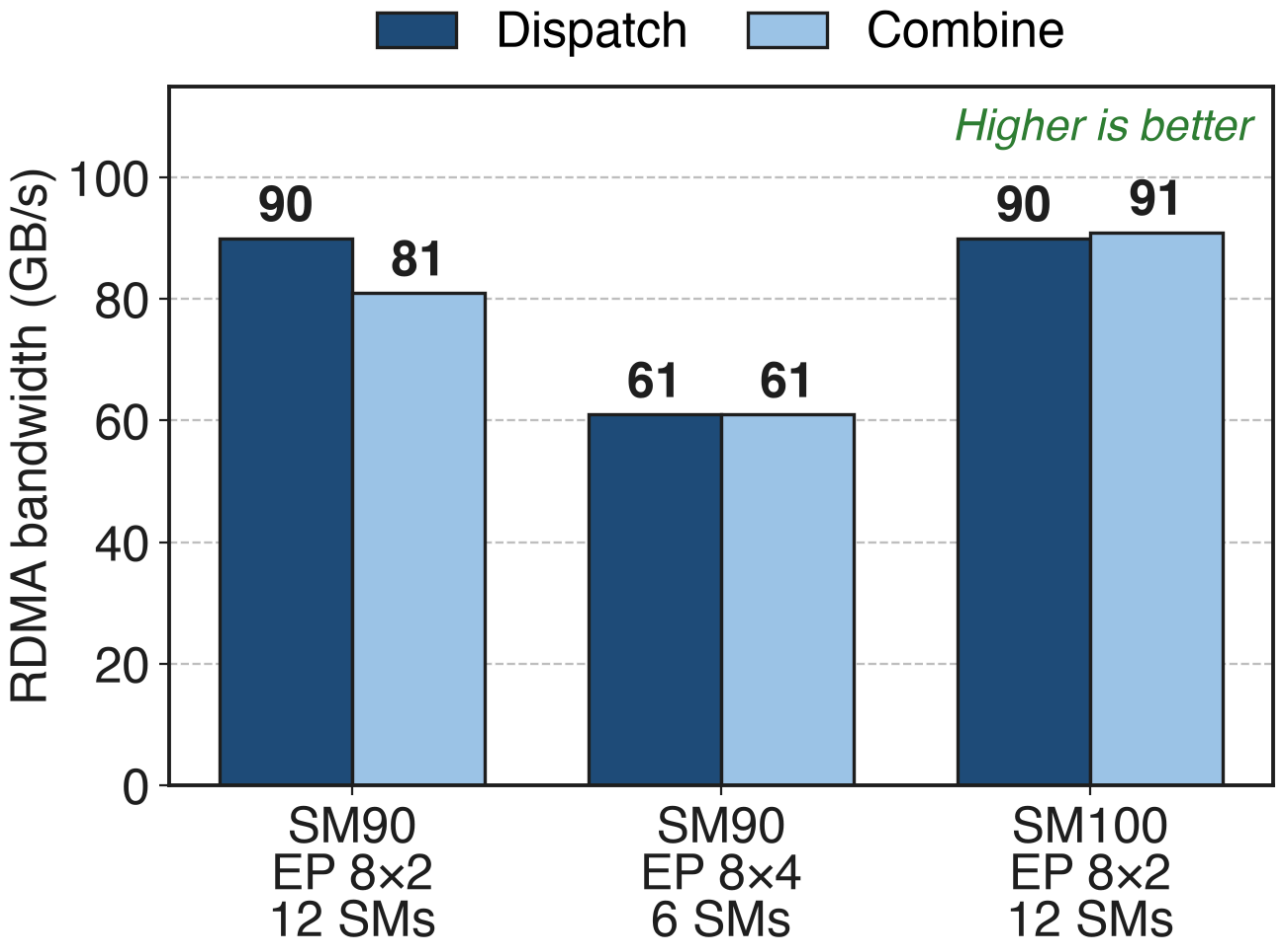
24 → 4–6

communication SMs per GPU for V3-like training, at equivalent or better performance

DeepEP V2 release notes, README at commit b306af0 and PR #605

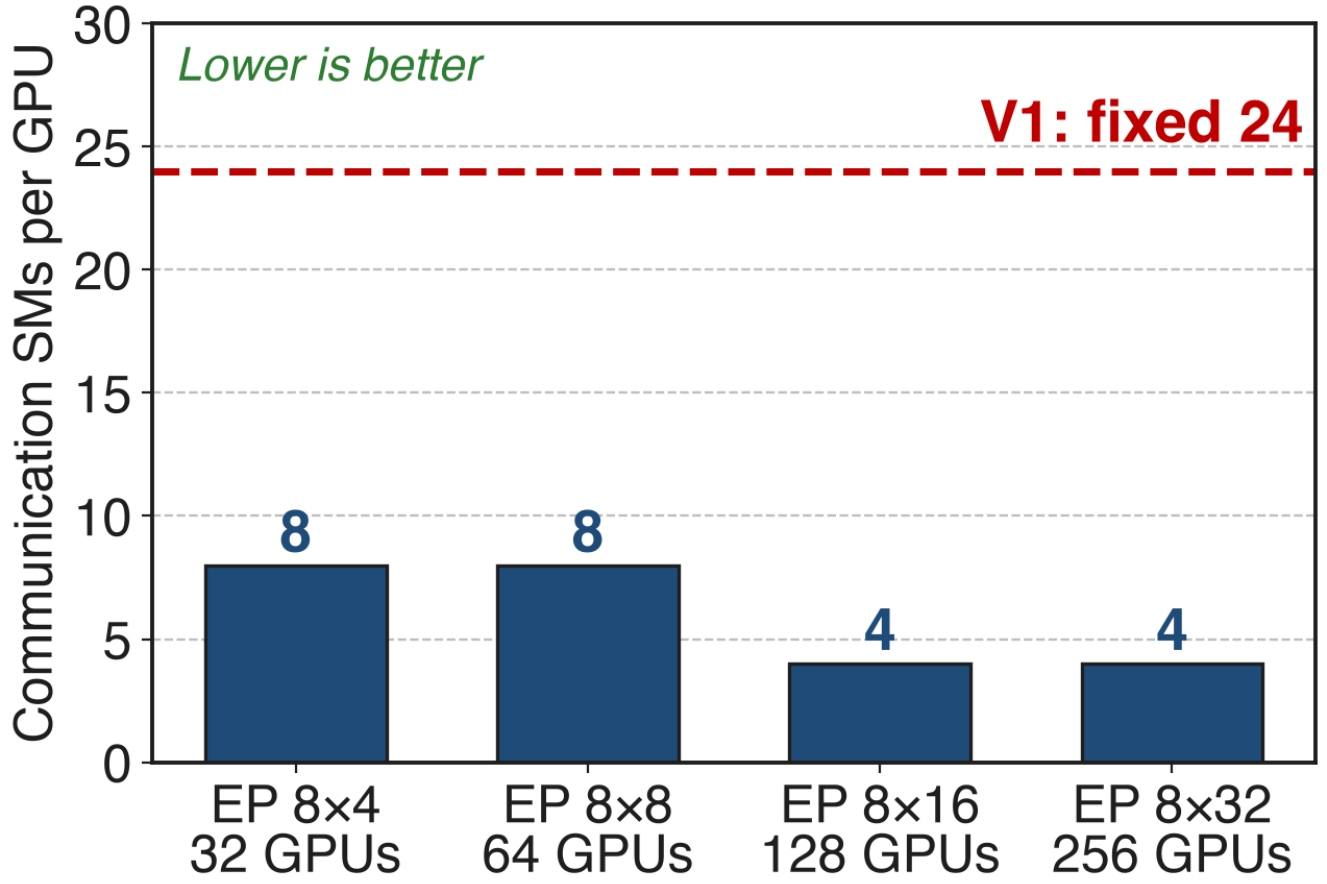
6 SMs sustain 61 GB/s at EP 8×4

- V3 setup: 8K tokens, hidden 7168, top-8
- FP8 dispatch, BF16 combine, CX7 NICs
- Logical bandwidth incl. local-rank traffic



Larger EP needs fewer SMs, never 24

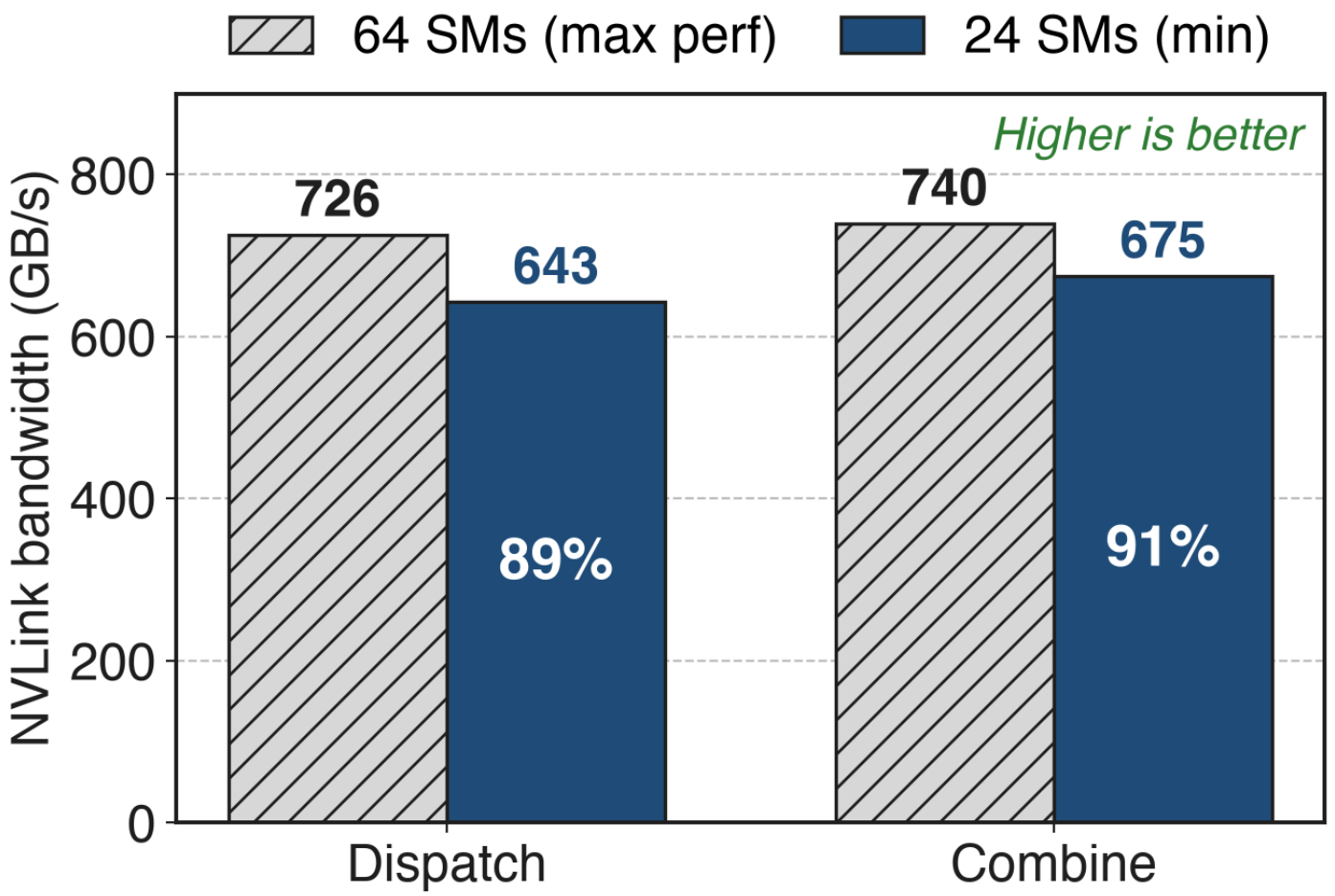
- More nodes per token, more RDMA bytes
- Lower token rate, fewer SMs to feed it



Analytic: DeepEP V2.5 formula; 256 experts, top-8, 50 GB/s RDMA; RDMA-bound at every size shown

24 SMs keep 89–91% of NVLink peak

- SM100, EP8 in one NVLink domain
- Overlap off: the model floors at 64 SMs
- Default: the small, overlap-friendly budget



The model trades generality for zero tuning

- **Balanced gate only**: not V3's group-limited gate
- Per-SM constants fixed; expand mode not modelled yet
- Bigger buffers than V1; `num_sms` override stays

Summary

- V1: **24 SMs** per GPU, a 10-size tuning table, two APIs
- **Insight**: SMs only feed the slowest link, so compute them
- One EPBuffer; SMs from the bottleneck; QPs from SMs
- **24** → **4–6** SMs; **6 SMs** → **61 GB/s** at EP 8×4

https://github.com/deepseek-ai/DeepEP